Rectangle Criteria for Girth Eight in Cyclic Lifted-Product Codes: A Lean-Verified Criterion and Decoding Case Studies

October 22, 2026; revised October 23, 2026

Abstract

Freij-Hollanti, Morris and Šapokaitė asked for sufficient conditions, expressed through 2×2 submatrices, for a cyclic lifted-product Tanner graph to attain its maximum possible girth of eight, and for an analysis of the resulting decoding-relevant properties. We give an exact criterion: individual rectangle voltages must be nonzero and specified pairs of rectangle voltages must not cancel. The characterization for the expanded binary matrix is proved in Lean. For the degree-three case we derive a bijection between eight-cycles and minimum $(4, 4)$ absorbing sets, a stopping set lower bound, and local syndrome bounds. A connected girth-eight family with conductance tending to zero shows that the criterion alone does not ensure uniform expansion. Exhaustive optimization at lift size fourteen reduces the absorbing-set count from 3,150 to the fixed-family minimum of 2,520 while preserving 252 physical and ten logical qubits. In an independent, paired min-sum experiment with 100,000 errors per noise level, one structural minimizer's recovery failure rate falls from 4.385% to 2.523% at physical error probability 4%. The girth criterion answers the definite mathematical part of the source question; the broader decoding program remains open. The benchmark does not isolate absorbing-set count as the cause of improvement: a new exhaustive low-weight census gives baseline distance six and distance at least eight for the other three codes. We distinguish Lean-checked theorems, written structural proofs, exact finite computations and statistical evidence.

1 The source question and the scope of our answer

In Section 5 of *Graphical Analysis of Lifted Product Code Constructions*, Freij-Hollanti, Morris and Šapokaitė explicitly ask for *"not just necessary, but sufficient conditions"* for the Tanner graph to have *"maximum possible girth of 8"*, together with the effects on other decoding-relevant graph properties [1]. The question concerns the cyclic, single-base-matrix lifted-product construction. It is presented as an open research question, rather than a numbered conjecture. Its first part asks for a definite mathematical criterion; its second part proposes a broader structural and experimental program.

We answer the first part by an equivalence in the entries of the original base matrix, not a test requiring construction of the much larger binary parity-check matrix. For the second part we establish specific consequences for degree-three graphs, a limitation on expansion, and a finite optimization and decoding study. These are partial answers to the broader research program, not a general performance characterization. Table 1 specifies the extent of each answer. The degree-three conclusions concern fully populated 3×3 bases; the girth theorem holds in every dimension with at least two rows and two columns.

Question	Answer	Evidence
Sufficient girth conditions	A necessary and sufficient rectangle-pair criterion	General Lean proof
Absorbing sets	In degree three, minimum sets are exactly the four-variable eight-cycle supports	Lean bijection; written $(4, 4)$ profile
Stopping sets and syndromes	Stopping size at least six; local cancellation bounds	Lean general lower bound and witness
Connectivity and expansion	Connectivity is separate; connected girth-eight graphs can have vanishing conductance	Written argument; finite checks
Finite design and decoding	Exact minimum in the specified normalized family; paired decoder comparison	Exhaustive search and Monte Carlo

Table 1: Completed girth criterion and partial answers to the broader decoding program, with evidence levels.

2 Predecessors and chronology

Fossorier's exponent-sum criterion supplies the classical short-cycle congruences [2]. Smarandache and Mitchell develop necessary and sufficient QC girth tests through matrix products and derived submatrices [3]. Our walk-based proof specializes the classical congruences and supplies the reduction from the lifted-product support to the original input matrix.

Raveendran and Vasić describe the product eight-cycle mechanism in hypergraph-product graphs [4]. Zhang's thesis studies cyclic cancellation and cycle counting for lifted products under additional base-factor girth hypotheses [5]. Raveendran, Declercq and Vasić exhibit the eight-label cancellation directly for the same monomial LP family [6]. Okada and Kasai use a related coordinate-support argument and a forced eight-cycle in square-base hypergraph products with constrained CPM lifts [7]. These works provide the closest mechanisms behind the proof below. Their stated settings and conclusions are distinguished from our entrywise equivalence in Table 2.

The absorbing-set design perspective is established in classical coding theory by Wang, Dolecek, Zhang and Wesel [8] and its cycle consistency matrix development [9]. McMillon, Beemer and Kelley give extremal absorbing-set bounds, including the relevant girth-eight degree-three lower bound [10]. Morris, Pllaha and Kelley develop the quantum absorbing-set framework and its decoder interpretation [11]. Our consequences apply this framework to the characterized LP family. Abiad and Castriota's later spectral treatment counts cycles and absorbing configurations and bounds stopping sizes in a different QC-GHP/GB setting [12]. The stopping bound also has a direct classical predecessor in Kelley, Sridhara, Xu and Rosenthal [16]. Representation-dependent BP behavior predates our intervention: Rigby, Olivier and Jarvis study augmented checks [17], and Miao et al. study overcomplete quantum check matrices [18]. Earlier trapping-set dynamics, including saturation and limit cycles, are studied by Tomasoni, Bellini and Ferrari [19]; the quantum symmetry/failure connection is already present in Raveendran and Vasić [4]. Our update equations follow the soft-syndrome min-sum framework [15], with the fixed arithmetic and parameters specified in the later reports.

The chronology separates earlier ingredients from an answer to the stated question. In particular, the roughly 21.9 years between Fossorier's criterion and the question are not a claim that the full LP equivalence was already stated in 2004. The present answer assembles those methods into one exact input-matrix theorem with an explicit formal certificate. Measured instead to this manuscript, the March 2025 direct LP predecessor is 591 days (1.62 years) earlier, the April 2026

Work	Date used	Δ years	Contribution and relationship
Fossorier [2]	August 2004	-21.9	Classical QC cycle congruences; ingredients for the input-matrix tests.
Wang et al. [8]	May 31, 2011	-15.03	Absorbing-spectrum design and measured classical decoding gains.
McMillon et al. [10]	March 31, 2021	-5.19	Early online publication of extremal absorbing-set bounds.
Smarandache and Mitchell [3]	May 7, 2021	-5.09	Classical necessary/sufficient QC girth conditions via matrix products.
Raveendran–Vasić [4]	October 14, 2021	-4.65	Product eight-cycle and quantum trapping-set analysis.
Zhang [5]	March 27, 2024	-2.20	Thesis defense; LP cyclic cancellation with additional hypotheses.
Raveendran et al. [6]	March 10, 2025	-1.25	Same-family LP cancellation and controlled base-matrix decoding comparisons.
Morris et al. [11]	March 3, 2026	-0.27	Published quantum absorbing-set framework; earlier preprints precede this date.
Okada–Kasai, v1 [7]	April 30, 2026	-0.11	Related square-base support argument and forced eight-cycle.
Freij-Hollanti et al. [1]	June 10, 2026	0	Explicit sufficient-condition and decoding-properties question.
Abiad–Castriota [12]	July 15, 2026	$+0.10$	Related spectral enumeration and stopping-bound program.
Okada–Kasai, v3 [7]	September 11, 2026	$+0.25$	Expanded finite-instance certificates in the square-base setting.
This manuscript/artifact	October 22, 2026	$+0.37$	Exact Lean characterization, structural consequences and finite benchmark.

Table 2: Signed chronology relative to June 10, 2026. A negative number means preceding the question. Day-resolved entries use elapsed days divided by 365.2425; the August 2004 entry is month-resolved and rounded to one decimal. The present artifact is dated 134 days after the question. Dates identify the versions or publication milestones used, not dates of private discovery.

square-base predecessor is 175 days (0.48 years) earlier, and the July 2026 spectral study is 99 days (0.27 years) earlier. The main answer here follows the published question by 134 days (0.37 years).

2.1 Later decoding-trajectory chronology and formal contributions

Table 3 extends the chronology beyond the original manuscript. Every follow-up listed was recorded on October 23, 2026: 135 days after the source question and one day after the original draft. The sequence is resolved by repository commits, not by assigning distinct calendar dates to work performed on the same day. The complete girth characterization is the earliest complete answer to that specific question known to us. For the follow-ups, the contribution column distinguishes concrete certificates and measurements from the established principles they instantiate.

Table 3: Later milestones, all October 23, 2026 (+135 days from the question). Links identify complete Lean files and reports at immutable revision `c556173`. Predecessor dates appear in Table 2 and the bibliography.

Milestone / commit	Contribution and relationship to earlier work	Evidence and complete Lean sources
Logical structure and distance controls `50ba916`, `feae8aa`, `e39a457`	Same local topology can have different logical roles. Exact distance audits identify a matched pair of $[[252, 10, 10]]$ codes; one million fresh paired errors distinguish their fixed-decoder recovery rates. Extends the design comparisons in [6] with explicit distance controls.	Logical/stabilizer certificate. Exact distances and sampling statistics are computational, not general Lean theorems. Distance audit; paired comparison.
Same-code representation intervention `8cf355c`, `7d58083`, `8e6efb6`	Changes absorbing counts while preserving the full code; a check merger also preserves variable degrees, connectedness and girth eight. Fresh paired tests find deterioration. Instantiates known representation dependence [17, 18]; altered check degrees/routing prevent isolating absorbing count.	Feasibility and performance are written/computational results. Later concrete matrix-kernel equalities check algebraic equivalence. Feasibility; million-error study.
Local message obstruction `4156628`	Derives the merger's message identity, sign reversal/attenuation cases and local acceptance obstructions. Explains specific harmful updates under the unchanged decoder, rather than proposing a new decoder.	General message lemmas; actual-code witnesses. Local certificates do not establish the aggregate penalty. Mechanism report.
Persistence and fresh holdout `888beeb`	Freezes mechanisms, then tests one million fresh paired errors per code. Proves finite rejection tubes and synchronized-flip parity preservation; a pinned-check certificate covers about 13–14% of syndrome failures. Relates to prior symmetry and min-sum trapping dynamics [4, 19].	Interval/persistence theory; parity lemma; four concrete tubes. Finite-horizon rejection, not infinite trapping or a full aggregate explanation. Holdout report.

Milestone / commit	Contribution and relationship to earlier work	Evidence and complete Lean sources
Complete trajectory comparisons `888beeb`	Joins initialization, exact updates and first-match stopping for two retained fresh examples: original exact recovery at updates 65/86; merged rejection through update 100, with identical syndrome solution sets.	Complete comparison theorems, with imported data/proof modules. Lean checks the mathematical decoder; compiler execution and noise sampling remain implementation bridges.
Entry, propagation and channel moments `c556173`	Proves the initial divergence criterion, propagation locality and concrete possible-influence cones reaching all 756 message coordinates by update four. Checks soft-only and delayed-entry examples and an exact second-update sign reversal; formalizes finite Bernoulli parity moments.	Initial rule; cones; complete entry examples; delayed activation; channel moments; signed moments. Full report.
Independent entry confirmation `c556173`	Another million fresh paired errors per code tests the initial-activity hypothesis. Soft-only entry partitions 451/985 and 571/1,210 net excess syndrome failures descriptively; baseline-hardness controls weaken the interaction.	Statistical/computational evidence, not a Lean FER theorem or causal mediation result. The full expectation-to-decoder bridge and aggregate failure explanation remain partial. Protocol, results and controls.

The formalization uses Lean `4.36.0-rc1` and Mathlib revision `5f7b4198d22a9c82e1c698a379334fe128095c82`. The original 75-report girth audit and 29-report structural audit are separate from the later trajectory modules; helper declarations are not counted as independent research answers. Appendix A and the online guide provide the complete code and reproducible checks. The original benchmark below remains its historical experiment; the later matched and same-code studies have distinct samples and are not pooled with it.

These follow-ups make the source's decoding-properties program more concrete, but do not establish a universally superior objective, causation by absorbing-set count alone, or a general error-floor formula. Earlier representation and trapping results motivate the analysis; our specific contribution is the fixed-code intervention, audited observations and checked mathematical trajectories. The long calendar gaps to general principles are not durations for which those principles remained unsolved.

3 Construction and notation

Let $r \geq 1$ and let $B = (B_{ij}) \in (\mathbb{Z}/r\mathbb{Z})^{m\times n}$, where $m, n \geq 2$. Every entry represents the circulant permutation $P^{B_{ij}}$; exponent zero represents an identity block. There are no missing input blocks. Write B^* for transposition followed by exponent negation. Expanding the block matrices

$$B_{\mathsf{X}} = [B \otimes I_n \mid I_m \otimes B^*], \qquad B_{\mathsf{Z}} = [I_n \otimes B \mid B^* \otimes I_m] \tag{1}$$

gives binary parity-check matrices $H_{\mathsf{X}}, H_{\mathsf{Z}}$ with $r(m^2 + n^2)$ columns and rmn rows. This is the source paper's Equation (2.3) [1], within the established lifted-product framework [13, 6].

Use checks $C(i, j, a)$ and variables $L(k, j, b)$, $R(i, \ell, b)$, with fibers $a, b \in \mathbb{Z}/r\mathbb{Z}$. Adjacency in G_{X}

is exactly

$$L(k, j, b) \sim C(i, j, b + B_{ik}), \tag{2}$$
$$R(i, \ell, b) \sim C(i, j, b - B_{\ell j}). \tag{3}$$

Left variables have degree m, right variables degree n, and checks degree $m + n$. Thus $m = n = 3$ gives a $(3, 6)$-biregular graph. Girth is the length of the shortest simple cycle; the explicit cycle below guarantees that the graphs considered here are not forests.

Define the rectangle and hexagon voltages

$$\mathcal{R}(a, b; c, d) = B_{ac} - B_{ad} + B_{bd} - B_{bc}, \tag{4}$$
$$\mathcal{Q}(a, b, c; d, e, f) = B_{ad} - B_{ae} + B_{be} - B_{bf} + B_{cf} - B_{cd}. \tag{5}$$

All arithmetic is in $\mathbb{Z}/r\mathbb{Z}$. Direct cancellation gives

$$\mathcal{Q}(a, b, c; d, e, f) = \mathcal{R}(a, b; d, e) + \mathcal{R}(b, c; d, f). \tag{6}$$

4 The exact girth-eight characterization

Theorem 4.1 (Rectangle-pair characterization)**.** *For the construction above, $g(G_{\mathsf{X}}) = 8$ if and only if both conditions hold:*

$$\mathcal{R}(a, b; c, d) \neq 0 \qquad \text{for } a \neq b,\ c \neq d; \tag{7}$$
$$\mathcal{R}(a, b; d, e) + \mathcal{R}(b, c; d, f) \neq 0 \qquad \text{for pairwise distinct } a, b, c \text{ and } d, e, f. \tag{8}$$

Equivalently, every eligible rectangle and every eligible hexagon voltage is nonzero. No connectedness, primality or normalization assumption is required.

The point of (8) is that nonzero individual rectangles do not prevent cancellation between two rectangles. The two conditions are tests on the original $m \times n$ matrix, including for the sparse product support of (1).

Lemma 4.2 (Four-cycle reduction)**.** *The graph G_{X} contains a four-cycle if and only if an eligible rectangle voltage of B vanishes.*

Proof. A four-cycle alternates between two variables and two checks. A variable and a neighboring projected check determine the variable fiber uniquely, and conversely a variable fiber determines the adjacent check fiber. Consequently distinct same-side vertices in this short cycle cannot have the same projection. Two left variables can share checks only when their second coordinates agree, yielding distinct row and column indices in B; closure is exactly (4). Two right variables give the transposed, negated criterion, which has the same zero set. A left and a right variable can share only the projected check at their common coordinate pair, and fiber uniqueness prevents them from sharing a second check. Thus a mixed four-cycle is impossible. Conversely, lifting a zero rectangle at any fixed unused coordinate produces four distinct vertices and hence a genuine four-cycle. □

Lemma 4.3 (Six-cycle reduction)**.** *The graph G_{X} contains a six-cycle if and only if an eligible hexagon voltage of B vanishes.*

Proof. In a simple six-cycle each pair among its three variables shares a check, and each pair among its three checks shares a variable. Fiber uniqueness therefore forces three distinct projections on each side.

The three projected checks form a triangle whose edges are labeled by the block type of the intervening variable. An L edge preserves the check's second coordinate; an R edge preserves its first coordinate. If both edge types occur, two edges share a type. Their coordinate equalities, combined with the third edge, identify two projected checks, a contradiction. Hence all three variables belong to the same block family. In the left family, a cycle can be written

$$L(d, j) \sim C(a, j) \sim L(e, j) \sim C(b, j) \sim L(f, j) \sim C(c, j) \sim L(d, j).$$

The fiber returns to its starting value exactly when $\mathcal{Q}(a, b, c; d, e, f) = 0$. The right family gives the transposed, signed version of the same collection of tests. Conversely, distinct row and column triples and a zero hexagon voltage lift to a simple six-cycle. □

The short-cycle closure calculations are the corresponding instances of Fossorier's QC criterion [2]. The coordinate reduction is related to the support argument of Okada and Kasai [7]; here it yields an equivalence for arbitrary fully populated rectangular exponent matrices.

Lemma 4.4 (Uniform eight-cycle)**.** *For every B with $m, n \geq 2$, the graph G_{X} contains a simple eight-cycle.*

Proof. Choose $i_0 \neq i_1$, $j_0 \neq j_1$, and arbitrary k, ℓ. The following vertices, listed in cycle order, satisfy (2)–(3):

$$\begin{array}{ll} C(i_0, j_0, B_{i_0 k} - B_{\ell j_0}), & L(k, j_0, -B_{\ell j_0}), \\ C(i_1, j_0, B_{i_1 k} - B_{\ell j_0}), & R(i_1, \ell, B_{i_1 k}), \\ C(i_1, j_1, B_{i_1 k} - B_{\ell j_1}), & L(k, j_1, -B_{\ell j_1}), \\ C(i_0, j_1, B_{i_0 k} - B_{\ell j_1}), & R(i_0, \ell, B_{i_0 k}). \end{array}$$

Distinct selected rows and columns give four distinct checks, two distinct left variables and two distinct right variables. The block families are disjoint. Thus the cycle is simple, regardless of its fiber values. □

This is the known LP cancellation mechanism [6], with explicit fibers and a separate simplicity check. Its product antecedents include [4, 5].

Proof of Theorem 4.1. Lemmas 4.2 and 4.3 equate the two voltage conditions with absence of all four- and six-cycles. Bipartiteness excludes odd cycles. Lemma 4.4 provides an eight-cycle, so the absence of shorter cycles is equivalent to girth eight. Finally, (6) rewrites the hexagon condition using only rectangle sums. □

Remark 4.5 (A useful failed shortcut). Over $\mathbb{Z}/3\mathbb{Z}$, the matrix $\left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 1 \end{smallmatrix}\right)$ has no zero eligible rectangle but does have a zero hexagon and a simple six-cycle. Lean verifies this obstruction. It refutes the rectangle-only shortcut; the source paper does not assert that shortcut.

5 Lean development and proof structure

The formal development uses Lean 4.36.0-rc1 and Mathlib revision `5f7b4198d22a9c82e1c698a379334fe128095c82`. The main girth library contains 75 audited theorem declarations. Separate consumers add three source-question corollaries, eight decoding declarations, three candidate certificates and 29 general structural declarations, for 118 audited declarations in these five groups.

These counts include supporting lemmas and certificates; they do not count 118 independent research results. The core and these four consumers occupy 2,064 source lines, including comments and module declarations. The additional checking consumer and prior-art comparisons are distributed as well.

The short final corollary is an application of this development. The substantive work establishes the relationship between the expanded binary matrix, lifted graph, explicit cycles, voltage obstructions and finite-set counting arguments. This section exposes the principal interfaces and selected proofs. Appendix A links the complete source at an immutable revision and provides reproduction commands. Every listing here is extracted from that source, with reversible TeX escapes for Unicode; `listings/excerpts.json` records its source file, line range and hash. A fragment is identified as such rather than presented as a standalone proof.

5.1 Principal theorem interfaces

In the girth library, I, J are row and column types and $r : \mathbb{N}$. The assumptions `[Nontrivial I]` and `[Nontrivial J]` mean that each type has at least two distinct elements. For the finite coding interpretation, use finite I, J and $r \geq 1$. The Lean theorem itself is more general. Its graph is Mathlib's `SimpleGraph`, and `girth` is the native graph invariant. The first complete declaration states both directions for the actual binary parity-check matrix:

Listing 1: The full expanded-matrix characterization and its final transfer proof.

```
theorem matrixTanner_hX_girth_characterization [Nontrivial I] [Nontrivial J]
    (B : I → J → ZMod r) :
    (matrixTanner (hX B)).girth = 8 ↔
      (∀ a b c d, a ≠ b → c ≠ d → baseRectangle B a b c d ≠ 0) ∧
      (∀ a b c d e f, a ≠ b → b ≠ c → a ≠ c → d ≠ e → e ≠ f → d ≠ f →
        baseHexagon B a b c d e f ≠ 0) := by
  rw [← xGraph_eq_matrixTanner]
  exact xGraph_girth_characterization B
```

The rewriting step uses the proved equality between the voltage-model graph and `matrixTanner (hX B)`. The called theorem contains the cycle analysis; its proof is not replaced by that rewrite. To obtain the source question's rectangle-only formulation, use `baseHexagon_eq_rectangle_add`. In particular, the sufficient condition is the following complete checked corollary:

```
theorem sufficient_rectangle_conditions [Nontrivial I] [Nontrivial J]
    (B : I → J → ZMod r)
    (hRect : ∀ a b c d, a ≠ b → c ≠ d → baseRectangle B a b c d ≠ 0)
    (hPairs : ∀ a b c d e f, a ≠ b → b ≠ c → a ≠ c →
      d ≠ e → e ≠ f → d ≠ f →
      baseRectangle B a b d e + baseRectangle B b c d f ≠ 0) :
    (matrixTanner (hX B)).girth = 8 := by
  apply (matrixTanner_hX_girth_characterization B).mpr
  exact ⟨hRect, by simpa only [baseHexagon_eq_rectangle_add] using hPairs⟩
```

For the structural results, let `E : V → C → Prop` be an arbitrary Tanner incidence relation, with decidable equality on both types and a finite check type C. The selected variable support `S : Finset V` is finite; the whole variable type need not be. `neighbors E v` contains all checks incident to v, and `selected E S c` contains the variables of S incident to c. `Absorbing E S` explicitly requires nonemptiness and strictly more even than odd induced-degree neighbors at every selected

variable. `Stopping E S` requires nonemptiness and at least two selected neighbors at every check touched by S.

`EightCycleOn E S T` supplies four distinct variables and four distinct checks, their alternating incidences, and the equalities identifying their supports with S, T. Thus it records an explicit simple cyclic ordering. The following complete declaration characterizes all absorbing supports of size at most four:

```
theorem absorbing_iff_cycle (S : Finset V) (hcard : S.card ≤ 4)
    (deg : ∀ v ∈ S, (neighbors E v).card = 3)
    (h4 : ¬HasFourCycle (bipartiteBase E))
    (h6 : ¬HasSixCycle (bipartiteBase E)) :
    Absorbing E S ↔ ∃ T, EightCycleOn E S T := by
  exact ⟨fun hs => absorbing_small_cycle E S hs hcard deg h4 h6,
    fun ⟨_,ht⟩ => cycle_absorbing E ht deg h4⟩
```

Here `HasFourCycle` and `HasSixCycle` are explicit cycle predicates on `bipartiteBase E`; the earlier library relates them to native simple closed walks. The degree assumption is imposed only on the selected variables. The two implications invoke the separately proved `absorbing_small_cycle` and `cycle_absorbing`, rather than postulating a classification of cycle supports.

The stopping theorem has a different degree hypothesis: it accepts any lower bound $d \geq 2$, with no upper bound on the size of S. Its complete signature is printed below; its proof continues in the linked source and is analyzed in Section 5.5.

Listing 2: Signature of the general stopping-set theorem; the body is in the full source.

```
theorem stopping_card_ge_twice_degree (S : Finset V) (hs : Stopping E S)
    (h4 : ¬HasFourCycle (bipartiteBase E))
    (h6 : ¬HasSixCycle (bipartiteBase E))
    (d : ℕ) (hd : 2 ≤ d) (hdegree : ∀ v ∈ S, d ≤ (neighbors E v).card) :
    2*d ≤ S.card
```

`stopping_minimum_six` specializes this result to degree three. The absorbing correspondence is strengthened from an existence statement to the explicit equivalence shown in Section 5.4.

5.2 Proof dependency diagram

Figure 1 records the mathematical dependencies behind the main interfaces. Each arrow means that the lower conclusion uses the results above it; routine helper lemmas and the import graph are omitted. In particular, the stopping bound is an independent general Tanner-graph argument, not a finite check on the selected modulus.

5.3 Constructing and checking the eight-cycle

The upper bound on girth is constructive. The complete definition below specifies all eight lifted vertices and discharges each adjacency by unfolding the incidence relation and normalizing additive expressions. Its fibers are exactly the ones in Lemma 4.4.

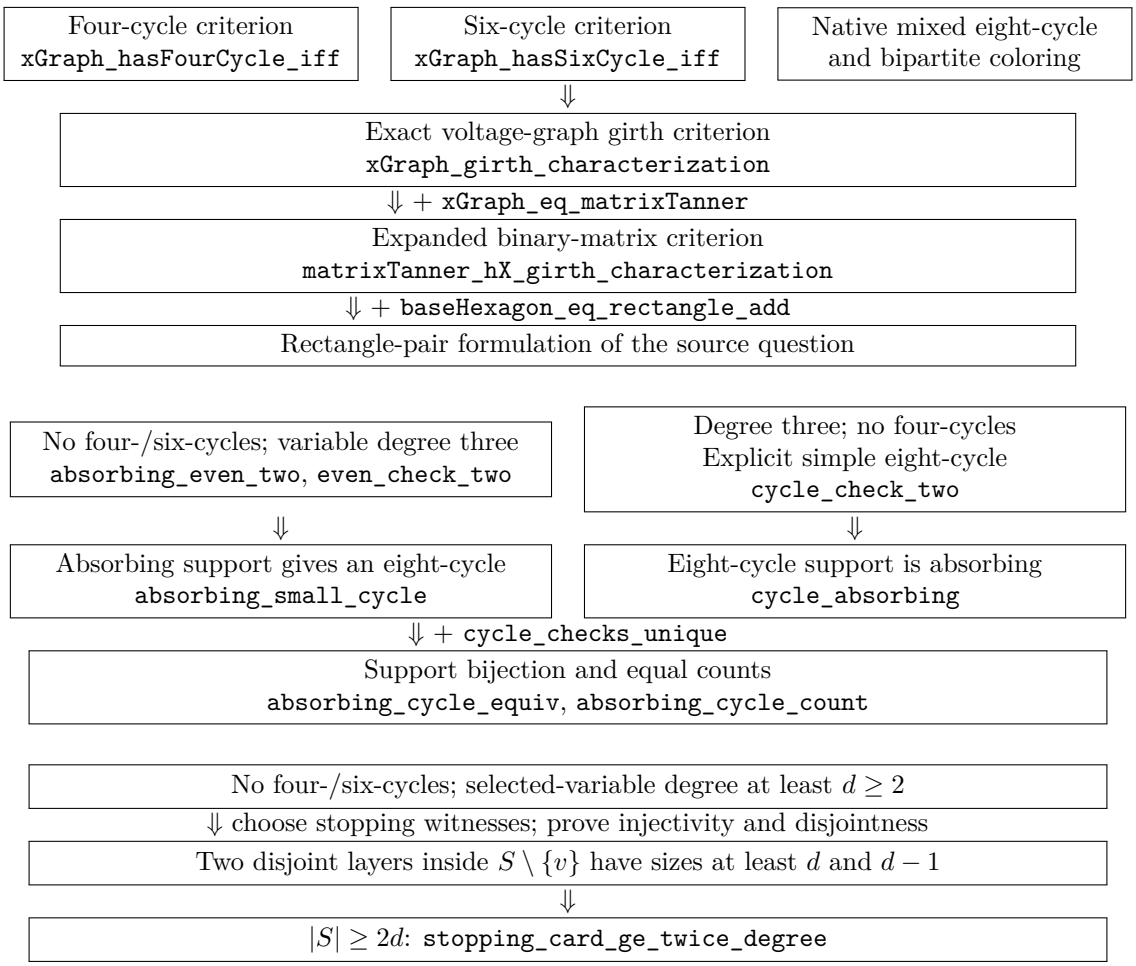


Figure 1: Dependencies of the girth characterization, absorbing-support bijection and stopping bound. The structural branches use generic Tanner incidence; application to our construction uses its girth and degree properties.

```
def mixedEight (B : I → J → ZMod r) (i0 i1 : I) (j0 j1 k : J) (l : I) :
    (xGraph B).Walk (.inr (i0,j0), B i0 k - B l j0)
      (.inr (i0,j0), B i0 k - B l j0) :=
  eightWalk
    (b := (.inl (.inl (k,j0)), -B l j0))
    (c := (.inr (i1,j0), B i1 k - B l j0))
    (d := (.inl (.inr (i1,l)), B i1 k))
    (e := (.inr (i1,j1), B i1 k - B l j1))
    (f := (.inl (.inl (k,j1)), -B l j1))
    (g := (.inr (i0,j1), B i0 k - B l j1))
    (h := (.inl (.inr (i0,l)), B i0 k))
    (by simp [xGraph, check_variable_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, variable_check_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, check_variable_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, variable_check_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, check_variable_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, variable_check_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, check_variable_adj, xSupport, xLabel]; abel)
    (by simp [xGraph, variable_check_adj, xSupport, xLabel]; abel)
```

Closure of a walk alone does not prove simplicity. The next complete proof uses distinct row and column choices to establish that the tail is a path, and then applies Mathlib's native cycle criterion:

```
theorem mixedEight_isCycle (B : I → J → ZMod r) (i0 i1 : I) (j0 j1 k : J) (l : I)
    (hi : i0 ≠ i1) (hj : j0 ≠ j1) : (mixedEight B i0 i1 j0 j1 k l).IsCycle := by
  rw [SimpleGraph.Walk.isCycle_iff_isPath_tail_and_le_length]
  constructor
  · simp [mixedEight, eightWalk, SimpleGraph.Walk.isPath_def, hj, Ne.symm hi, Ne.symm hj]
  · change 3 ≤ 8; omega
```

The companion length theorem is by reflexivity. Together these establish a native simple cycle of length eight, not only an arithmetic cancellation or a closed walk that might revisit a vertex. The four-/six-cycle classification and bipartite coloring then give the matching lower bound.

5.4 The absorbing-support bijection

The forward construction first proves that each selected degree-three variable has at least two even check neighbors. For $|S| \leq 4$, each such check has exactly two selected neighbors: a degree-four check would prevent a second even check by the four-cycle exclusion. Starting from one variable, two such checks give two distinct neighbors. A second check at one neighbor gives a fourth variable; equality with an earlier variable would create a four- or six-cycle. This constructs the required ordering. The converse checks the induced multiplicities of the cycle checks and verifies the even/odd inequality directly.

For uniqueness, `cycle_shared_check` proves that a check meeting two selected cycle variables is a cycle check. A consecutive pair has a unique common check by four-cycle exclusion; an opposite pair would close a six-cycle. Hence `cycle_checks_exact` identifies the cycle's check support with the checks having at least two selected neighbors. This proves `cycle_checks_unique`.

`UnorientedEightCycles E` is the subtype of variable/check support pairs equipped with an `EightCycleOn` certificate. Its cyclic ordering is existential data in a proposition, so choosing a different origin or direction does not create a new element. The target `FourAbsorbingSupports`

`E` is the subtype of finite supports with cardinality four and the absorbing property. Here is the complete bijection construction, including both injectivity and surjectivity:

```
noncomputable def absorbing_cycle_equiv
    (deg : ∀ v, (neighbors E v).card = 3)
    (h4 : ¬HasFourCycle (bipartiteBase E))
    (h6 : ¬HasSixCycle (bipartiteBase E)) :
    UnorientedEightCycles E ≃ FourAbsorbingSupports E := by
  let f : UnorientedEightCycles E → FourAbsorbingSupports E := fun p =>
    ⟨p.val.1, cycle_card_four E p.property,
      cycle_absorbing E p.property (fun v _ => deg v) h4⟩
  apply Equiv.ofBijective f
  constructor
  · intro p q hpq
    have hS : p.val.1 = q.val.1 := congrArg Subtype.val hpq
    have hT : p.val.2 = q.val.2 := cycle_checks_unique E p.property
      (hS.symm ▸ q.property) h4 h6
    exact Subtype.ext (Prod.ext hS hT)
  · intro S
    obtain ⟨T,hT⟩ := absorbing_small_cycle E S.val S.property.2
      S.property.1.le (fun v _ => deg v) h4 h6
    exact ⟨⟨(S.val,T),hT⟩, Subtype.ext rfl⟩
```

The function `f` forgets the check support. Its injectivity uses uniqueness of that support, while surjectivity uses the explicit forward cycle construction. `absorbing_cycle_count` transfers the equivalence to equality of cardinalities. Independently, `cycle_native` constructs a native Mathlib simple closed walk of length eight from every cycle certificate. The formal representation is therefore connected to the graph semantics used in the girth theorem.

5.5 Stopping witnesses: disjointness and cardinality

The stopping proof begins with $v \in S$ and a check c_0 of v. For every check c of v, choose another selected variable $w(c)$. Put $a = w(c_0)$. For every check $c \neq c_0$ of a, choose another selected variable $z(c)$. Four-cycle exclusion makes each witness map injective and excludes $z(c) = v$. The difficult remaining collision is $z(c) = w(e)$. The following contiguous excerpt is the entire local proof excluding that collision; `hwE`, `hzE` record the chosen incidences, and `hwne`, `hzne` record distinctness from the respective parent variables.

```
have znw : ∀ c e, z c ≠ w e := by
  intro c e h
  have hca := (mem_neighbors E a c.val).mp (Finset.mem_erase.mp c.property).2
  have hc0ne := (Finset.mem_erase.mp c.property).1
  have hwe := (mem_neighbors E v e.val).mp e.property
  have he0 : e.val ≠ c0 := by
    intro he
    have heq : e = c0' := Subtype.ext he
    exact hzne c (h.trans (congrArg w heq))
  have hce : c.val ≠ e.val := by
    intro he
    exact hc0ne (check_unique E h4 hav hca (he ▷ hwe) hac0 hvc0)
  have hawe : a ≠ w e := by
    intro he
    exact he0 (check_unique E h4 hav (he ▷ hwE e) hwe hac0 hvc0)
  exact six_contradiction E h6 hav.symm hawe (hwne e).symm
    (Ne.symm hc0ne) hce he0.symm hvc0 hac0 hca (h ▷ hzE c) (hwE e) hwe
```

The intermediate equalities would create four-cycles; after excluding them, the final invocation exhibits the forbidden six-cycle. The proof then forms the two finite images, `first` and `second`, proves they are disjoint, and places their union in `S.erase v`. These injectivity, disjointness and inclusion facts justify the final cardinality calculation, reproduced verbatim:

```
have hcard := Finset.card_le_card hsub
rw [Finset.card_union_of_disjoint hdis, hf, hsnd,
  Finset.card_erase_of_mem ((mem_neighbors E a c0).mpr hac0),
  Finset.card_erase_of_mem hv] at hcard
have hda := hdegree a haS
omega
```

Thus the arithmetic tactic `omega` finishes only after the graph argument has supplied the two disjoint layers. The calculation is $|S| - 1 \geq \deg(v) + \deg(a) - 1 \geq 2d - 1$, yielding $|S| \geq 2d$.

5.6 Trust boundary and scope

All 118 audited declarations use only `propext`, `Classical.choice` and `Quot.sound`. There are no admitted proofs or native evaluation axioms. Classical choice selects witnesses; Lean's kernel checks the proof terms produced by tactics such as `simp`, `abel` and `omega`. The source and logs separate the core girth audit from the four supplementary groups.

The absorbing-support characterization, actual support bijection, count equality, minimum-size exclusion and general stopping inequality are Lean results. The elementary $(4, 4)$ odd-check count, companion-stabilizer identification and conductance argument retain written proofs. The finite optimization and decoding statistics have computational audits rather than Lean proofs. Appendix A makes the complete formal proofs and those evidence boundaries directly discoverable.

6 Consequences for degree-three decoding graphs

Fix $m = n = 3$ and assume the conditions of Theorem 4.1. For a variable set S, induced check degree means degree in the graph on $S \cup N(S)$, including every edge from a selected variable.

Definition 6.1. An (a, b) *absorbing set* is a nonempty set of a variables whose induced graph has b odd-degree checks, such that every selected variable has strictly more even-degree than odd-degree

check neighbors. It is *elementary* if every neighboring check has induced degree one or two. A *stopping set* is a nonempty variable set with no neighboring check of induced degree one.

These conventions agree with the source and quantum absorbing-set framework [1, 11]. The size lower bound in the next result is consistent with the established extremal bounds of [10].

Theorem 6.2 (Minimum absorbing sets)**.** *In any variable-degree-three Tanner graph of girth eight, no absorbing set has fewer than four variables. Its four-variable absorbing sets are exactly the supports of simple eight-cycles, with one such cycle per support. All are elementary* $(4, 4)$ *sets. Consequently their multiplicity equals the number of eight-cycles.*

Proof. A selected variable needs at least two even check neighbors. For at most three selected variables, each such check has degree two. No pair of variables can share two checks, as that would make a four-cycle. Sets of size one or two therefore fail absorption. A size-three set would need three pair checks forming a six-cycle, also excluded.

Take the four variables of an eight-cycle. An additional edge from one of its checks to another selected variable creates a four-cycle. An outside check meeting two selected variables creates a four-cycle when they are consecutive, or a six-cycle when they are opposite. Thus the four cycle checks have induced degree two and the four remaining neighbors are distinct checks of degree one. Each variable has two even neighbors and one odd neighbor.

Conversely, an absorbing set of four variables cannot meet a degree-four check: every other even check would share a pair with it and create a four-cycle, leaving each variable at most one even neighbor. Nor can it meet a degree-three check: each of that check's variables could then have at most one even neighbor, paired with the fourth variable. Hence all check degrees are at most two. Replace every degree-two check by an edge between its variables. The resulting normal graph is simple, triangle-free, has four vertices and minimum degree two. It is exactly a four-cycle. This also proves uniqueness of the corresponding Tanner eight-cycle. □

Lean proves the support characterization, exclusion of smaller nonempty absorbing sets, and a genuine equivalence between four-variable absorbing supports and unoriented eight-cycles. A cycle is represented by its variable and check supports with an explicit simple cyclic ordering; rotations and reversals are not counted separately. The development constructs a native Mathlib simple closed walk of length eight from each certificate and proves that the variable support uniquely determines the check support. The 4 odd-check count in the elementary $(4, 4)$ description is established by the written incidence argument above.

The conclusion is a sharp threshold, not removal of absorbing sets: Lemma 4.4 makes four-variable sets unavoidable in this model. It converts the next design objective into eight-cycle minimization.

Proposition 6.3 (Stopping size and a sharp example)**.** *A Tanner graph of girth at least eight and minimum variable degree* $d \geq 2$ *has stopping-set size at least* $2d$*. For the normalized* 3×3 *constructions here, this bound is attained at six.*

Proof. Choose a variable v in a stopping set and d of its check neighbors. Each check needs a second selected variable. These d witnesses are distinct, since a coincidence creates a four-cycle. Choose one witness a. At least $d - 1$ other checks of a each need another selected variable. The new witnesses are distinct and cannot equal v by the four-cycle exclusion; none can be a different first-level witness by the six-cycle exclusion. There are at least $1 + d + (d - 1) = 2d$ variables.

If the first row and column of B vanish, select

$$S_6 = \{L(0, j, 0) : 0 \leq j < 3\} \cup \{R(i, 0, 0) : 0 \leq i < 3\}. \tag{9}$$

Exactly the nine checks $C(i, j, 0)$ meet this set, each twice. It is a stopping set of size six and has zero syndrome. □

The general inequality is checked by `stopping_card_ge_twice_degree`; its hypotheses explicitly exclude four- and six-cycles, require a nonempty stopping set, and impose the degree lower bound only on its selected variables. The corollary `stopping_minimum_six` specializes it to degree three.

The same support is a row of H_{Z}, indexed by $(0, 0)$ and fiber zero. It represents a stabilizer, so this fact does not assign quantum logical distance six. For the binary classical kernel of H_{X}, the stopping lower bound and (9) do give minimum nonzero weight six. Lean checks the concrete induced-degree profile over $\mathbb{Z}/7\mathbb{Z}$.

Proposition 6.4 (Small-error syndrome bounds)**.** *In a four-cycle-free Tanner graph, errors at two distinct variables v, w produce at least $\deg(v) + \deg(w) - 2$ odd checks. In degree three and girth at least eight, one, two and three errors expose at least three, four and five degree-one checks, respectively.*

Proof. Distinct variables share at most one check. The two-error syndrome is $N(v) \triangle N(w)$, proving the first assertion; this argument is formalized in Lean. For S with $a \leq 3$ variables, the induced incidence graph is a forest, since any cycle would have length at most six. If $E = \sum_{v \in S} \deg(v)$ and there are $c \geq 1$ components, then $|N(S)| = E - a + c \geq E - a + 1$. If t checks have degree one, $E \geq t + 2(|N(S)| - t)$, whence $t \geq E - 2a + 2$. Set $E = 3a$. □

These are graph and syndrome statements. Failure-inducing sets also depend on decoder dynamics, as emphasized by [4, 11]. Section 9 tests that distinction directly.

7 Connectivity and a limitation on expansion

Girth eight alone does not ensure connectedness. For example, doubling every exponent of the modulus-seven matrix below and working modulo fourteen yields two components, because every edge preserves fiber parity. Direct BFS gives two components of 189 vertices each.

Theorem 7.1 (Connected girth eight with vanishing conductance)**.** *Let $r = 7k$, $k \geq 1$, and interpret the fixed integer matrix*

$$B_0 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 3 & 6 \end{pmatrix} \tag{10}$$

modulo r. The corresponding graphs are connected and have girth eight. For $k \geq 2$, their edge conductance satisfies

$$\phi(G_{\mathsf{X}}) \leq \frac{4}{3\lfloor r/2 \rfloor} \longrightarrow 0. \tag{11}$$

Proof. All eligible rectangle and hexagon sums for B_0 are nonzero modulo seven, as checked by Lean in the modulus-seven case. If any vanished modulo $7k$, it would vanish modulo seven. Theorem 4.1 thus gives girth eight for every k. The upper-left rectangle has voltage one, which generates the whole cyclic group. The source's connectivity criterion (Theorem 3.6 of [1]) therefore gives connectedness.

Let S contain all vertex types with fibers $0, \ldots, \lfloor r/2 \rfloor - 1$. Define

$$\phi(G_{\mathsf{X}}) = \min_{\varnothing \neq T \neq V} \frac{|\partial T|}{\min(\mathrm{vol}(T), \mathrm{vol}(V \setminus T))}.$$

There are nine degree-six checks and eighteen degree-three variables per fiber, or 108 units of volume. For $k \geq 2$, both fiber arcs have length at least seven. Each matching of displacement ϵ, where $|\epsilon| \leq 6$, crosses the cut in exactly $2|\epsilon|$ edges. The sum of the entries of B_0 is 12; each appears in three left and three right edge types. Thus $|\partial S| = 2 \cdot 6 \cdot 12 = 144$, and the smaller volume is $108\lfloor r/2 \rfloor$. This proves (11). □

The cuts at $r = 14, 28, 56$ have conductance ratios 0.190476, 0.095238 and 0.047619, respectively, confirmed by direct enumeration. These are upper bounds from specified cuts. They show that the girth criterion alone cannot provide a uniform expansion guarantee. Expansion remains a separate design objective, as in the source's hierarchy [1] and the related spectral program [12].

8 Exhaustive finite optimization

Fix lift size $r = 14$ and normalize the first row and column to zero:

$$B(a, b, c, d) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & a & b \\ 0 & c & d \end{pmatrix}. \tag{12}$$

Girth eight forces each interior entry to be nonzero, leaving $13^4 = 28,561$ candidates. Applying the exact rectangle/hexagon tests and the connectivity condition $\gcd(14, a, b, c, d) = 1$ leaves 10,368 bases. In this normalized model the four interior exponents generate the cycle-voltage subgroup, which explains the gcd test.

The search partitions these bases into 25 orbits under row/column permutations, normalization, multiplication by units modulo fourteen, and transposition. For the gauge change $B'_{ij} = B_{ij} + u_i + v_j$, the graph isomorphism shifts fibers of $C(i, j)$ by $u_i - v_j$, of $L(k, j)$ by $-v_k - v_j$, and of $R(i, \ell)$ by $u_i + u_\ell$. Transposition exchanges block families, swaps check coordinates and negates fibers. These identities justify counting a representative of every orbit. The number 25 refers to this explicit symmetry group, rather than an assertion that no further graph isomorphisms exist.

Each representative is checked for connectedness by BFS. Eight-cycle counts are computed both by check-projection enumeration with four distinct variable labels and by an independent depth-eight traversal of the Tanner graph. The validation script compares both algorithms on all 25 representatives. It also checks explicit graph bijections for each symmetry type and repeats the search partition.

Proposition 8.1 (Computed fixed-size optimum)**.** *In the normalized connected girth-eight search at $r = 14$, the minimum number of four-variable absorbing sets is 2,520. Exactly 1,728 bases attain it, forming four orbits under the stated symmetries.*

Proposition 8.1 is established by exhaustive finite computation; it is not a Lean optimality theorem. The four representative interiors are

$$(1, 2, 4, 11), \quad (1, 3, 6, 10), \quad (1, 3, 7, 4), \quad (1, 4, 6, 2).$$

The first and third encode eight logical qubits. The second and fourth preserve the baseline's ten and are the primary rate-matched candidates:

$$B_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 3 \\ 0 & 6 & 10 \end{pmatrix}, \qquad B_4 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 4 \\ 0 & 6 & 2 \end{pmatrix}. \tag{13}$$

For both, $\operatorname{rank}(H_X) = \operatorname{rank}(H_Z) = 121$ and $K = 252 - 121 - 121 = 10$, by exact binary elimination. Lean independently verifies their girth eight. Their count is 20% below the baseline's 3,150 and 4.76% below the previous candidate $B(1, 2, 3, 8)$ with 2,646. Equal length and dimension do not imply equal quantum distance.

9 Measured decoding performance

9.1 Protocol and independent checks

We use syndrome normalized min-sum with a flooding schedule, normalization $\alpha = 0.8$, at most 100 iterations and message saturation at absolute LLR 50. The prior is $\log((1-p)/p)$. The implementation uses fixed-point messages with twenty fractional bits, rounds normalization magnitudes to the nearest integer with ties up, assigns positive sign to zero messages and hard bit zero to a zero posterior. It halts at the first syndrome match, including the initial zero estimate. There is no post-processing or per-design tuning. The syndrome min-sum method and LP simulation framework have prior treatments in [14, 15]; our fixed implementation is specified to make this comparison reproducible.

Errors are independent Bernoulli(p) in one Pauli sector, with perfect syndrome measurements. The decoder receives the syndrome and prior. The true error and H_Z are used only afterward for scoring. If $\hat{e}$ is the correction, recovery succeeds exactly when

$$e + \hat{e} \in \operatorname{row}_{\mathbb{F}_2}(H_Z). \tag{14}$$

Thus nonzero stabilizer residuals count as success; syndrome mismatch or a nonstabilizer residual counts as failed recovery. This is the standard distinction between exact recovery, degeneracy and logical miscorrection [11]. The experiment is a one-sector code-capacity comparison, not a circuit-level or depolarizing-channel benchmark.

An independent Python implementation evaluates the direct extrinsic message equations and matches the C++ engine on 128 small cases, including an exact repetition-code oracle. Every primary code is checked on all single-qubit errors, zero error, an injected stabilizer and a constructed zero-syndrome logical operator. These checks distinguish the two cases in (14). All CSS row-pair orthogonality relations and GF(2) ranks are checked independently.

An exploratory experiment used 10,000 shared errors at each of five probabilities with seed 20261022. The confirmation uses a new seed, 20261023, and 100,000 independent errors at each of $p = 0.01, 0.02, 0.04$. At each probability all four rate-matched codes receive the same error patterns. The confirmation therefore comprises 300,000 independent patterns and 1.2 million decoder runs. Both rate-preserving structural minima were retained; selection did not use the confirmation outcomes.

9.2 Results

At $p = 4\%$, B_2 has approximately 42.46% fewer failed recoveries than B_0. Wilson 95% marginal intervals are $[4.260\%, 4.514\%]$ and $[2.428\%, 2.622\%]$, respectively. Using the shared errors, the paired difference is -1.862 percentage points, with approximate pointwise 95% interval $[-2.011, -1.713]$. At $p = 2\%$, it is -0.118 points with interval $[-0.166, -0.070]$. At $p = 1\%$ there are only 33 baseline and 36 B_2 failures; the paired interval includes zero, so this experiment does not establish improvement there.

Code	$(4,4)$ sets	$p = 1\%$	$p = 2\%$	$p = 4\%$
$B_0 = B(1,2,3,6)$	3,150	0.033%	0.369%	4.385%
$B(1,2,3,8)$	2,646	0.031%	0.278%	2.750%
$B_2 = B(1,3,6,10)$	2,520	0.036%	0.251%	2.523%
$B_4 = B(1,4,6,2)$	2,520	0.035%	0.278%	2.607%

Table 4: Recovery failure rates, 100,000 trials per cell. Every code has 252 physical qubits, ten logical qubits, connectedness and girth eight; quantum distance is not matched (Section 9.4).

The two minima are not clearly distinguished: at $p = 4\%$ their paired difference is -0.084 points, with interval $[-0.208, +0.040]$. Intervals are pointwise and not corrected for multiple comparisons. We infer neither an asymptotic threshold nor a low-noise error-floor law from these measurements.

Code	Syndrome mismatch	Matched logical failure	Stabilizer successes*
B_0	2,981	1,404	5,834
$B(1,2,3,8)$	2,714	36	5,849
B_2	2,503	20	6,005
B_4	2,565	42	6,031

Table 5: Outcome decomposition at $p = 4\%$, per 100,000 trials. *Successful nonzero stabilizer residuals, in addition to exact error recovery.

The improvement includes a large reduction in logical miscorrections. Mean iterations, counting failures at the limit, decrease from 9.689 for B_0 to 8.857 for B_2. Absorbing-set count is a structural selection objective; the exponent changes also affect algebraic and other graph properties. The census in Section 9.4 gives baseline distance six and distance at least eight for the other codes. Thus length and rate matching must not be read as distance matching. The experiment establishes an outcome for these codes, channel and decoder, not an isolated causal effect of absorbing-set count or a universal objective ranking.

9.3 Exhaustive targeted errors and the earlier design framework

We additionally decode every exact four-variable absorbing support, with no other erroneous variables, at each of $p = 0.01, 0.02, 0.04$. Every support is checked directly for its $(4,4)$ profile. The baseline fails on 378 of 3,150 supports, or 12%, always through a matched nonstabilizer residual. The previous best and both minima succeed on every support tested. Thus an absorbing support is not automatically failure-inducing for this min-sum decoder.

All failing baseline supports are mixed. Their residuals have weight six in 294 cases and weight eight in 84 cases. Explicit support witnesses are checked for zero syndrome and exclusion from $\mathrm{row}(H_{\mathsf{Z}})$. The weight-six witness bounds the baseline's distance in the tested Pauli sector by six. The exhaustive census below sharpens the baseline distance to six and proves a lower bound of eight for each other code; it does not assign an exact optimized distance. Under Bernoulli noise, these disjoint exact supports contribute $378p^4(1-p)^{248}$ to the baseline's failure probability; they are one error subclass, not an estimate of the full failure rate.

Raveendran, Declercq and Vasić already demonstrate that different base matrices can produce different decoding performance at equal length, rate and girth eight [6, Sec. V-A, Eq. (92), Fig. 1]. Their comparison uses 600-qubit codes and BP+OSD-10. Our baseline also satisfies their column-partition pattern $c_0 + c_2 = 2c_1$ modulo fourteen; the previous best and both minima avoid every

corresponding three-column or three-row relation. This links our example to their low-weight-logical design framework without assigning an optimized distance or claiming a new decoder. The finite optimum and the specified min-sum benchmark are the concrete contributions of this study.

9.4 Low-weight logical census and a limit of local topology

The source's conclusion specifically asks how row/column constraints producing low-weight logical operators translate into Tanner-graph properties [1, 6]. We address a finite part of that question using the same four benchmark codes, with no new noise or decoder settings. For each Pauli sector, distinct nonzero column syndromes and distinct unordered-pair syndromes exclude kernel words of weights two and four. Odd column degree three excludes all odd-weight kernel words. These checks exclude every nonzero kernel word below weight six.

We enumerate all $\binom{252}{3} = 2{,}635{,}500$ column triples per sector, sort by exact syndrome, and join equal-syndrome triples. Lower-weight exclusion guarantees disjointness of distinct triples in a bucket. Every weight-six kernel support has exactly ten unordered $3+3$ partitions. Each resulting support is tested against the full companion stabilizer row span. Both sectors give the following identical counts:

Code	Weight-six stabilizers	Weight-six logical supports	Quantum distance
B_0	126	14	6
$B(1,2,3,8)$	126	0	≥ 8
B_2	126	0	≥ 8
B_4	126	0	≥ 8

Table 6: Exhaustive finite census, per Pauli sector. A lower bound of eight is not an assertion of exact distance eight.

This identifies a material confound in the interpretation of Table 4: the better-performing codes also exclude the baseline's minimum-weight logical class. The experiment cannot separate that algebraic change from the absorbing-count reduction. The census supports a specific design check—exclude low-weight nonstabilizer kernel supports—but does not prove that this check alone predicts the measured failure rates.

There is a concrete limit to a purely local graph description. With the artifact's zero-based column indexing, the baseline support $\{28, 56, 84, 126, 184, 244\}$ has zero syndrome and is not a stabilizer. The support $\{0, 14, 28, 126, 168, 210\}$ is a companion stabilizer row. Both induced Tanner subgraphs consist of six variables and nine checks of degree two; contracting the checks gives $K_{3,3}$ in both cases. Thus their local unlabeled topology agrees, while their quantum roles differ. A test using only that topology cannot distinguish these two supports; the global companion row space is essential.

The supplementary Lean file `CyclicLogicalSix.lean` checks the actual circulant expansion, both local adjacency certificates and a dual linear functional that annihilates every companion row but evaluates to one on the logical support. Linearity proves nonmembership in the entire stabilizer span. Its ten audited theorem declarations are supplied in the source bundle. The exhaustive distance lower bounds and census remain finite computational evidence; they are not formalized by this small certificate. The historical frozen revision linked in Appendix A covers the earlier girth/structural development; this new certificate is included in the updated bundle.

The partition mechanism and low-weight-logical design motivation are due to Raveendran, Declercq and Vasić [6]. Our finite census and checked same-topology/different-role example apply

that research direction to these benchmark instances; no priority for the general mechanism is claimed.

10 Reproducibility and remaining scope

The companion artifact is organized under `formalization/cyclic-lifts` in the anonymous project repository `https://github.com/alectionary/vibe-math`. The manuscript package contains a snapshot of all Lean sources, the pinned Lake configuration, experiment programs, JSON results, encoded paired outcomes and validation scripts needed for this paper. Its manifest records SHA-256 hashes; the snapshot does not depend on a moving branch or unpublished working tree. Build instructions are included with the package.

The library and consumers verify the main theorem and concrete certificates using the pinned Lean/Mathlib environment. The standard-library Python search and C++17 decoder reproduce the finite experiments. The statistical audit reconstructs counts and paired comparisons from stored per-trial flags. No physical experiments were performed.

The precise girth-condition question is settled by Theorem 4.1. Its decoding-properties continuation receives three partial answers: structural consequences, a counterexample to uniform expansion from girth alone, and an optimized finite code with a measured recovery improvement at the specified noise levels. These results do not complete the source's broader decoding-properties program. A general classification across arbitrary degrees, exact optimized quantum distances above the new lower bound, optimization over lift sizes, and comparisons across decoders and noisy measurement models remain separate tasks. The absorbing-support bijection and stopping lower bound are now Lean-checked general theorems. The elementary odd-check count, the companion-stabilizer identification and conductance argument remain available for further formalization.

Acknowledgments and research provenance. The question is credited to Ragnar Freij-Hollanti, Kirsten D. Morris and Patricija Šapokaitė; their structural program guided the choice of targets. The predecessor contributions credited in Section 2 guided the proof comparison and interpretation of decoding outcomes. OpenAI Codex was used for literature discovery, mathematical exploration, Lean development, experiment implementation and manuscript drafting.

A Complete Lean proof and reproduction guide

The PDF prints selected declarations and substantive excerpts; the full formal proof, including every helper lemma, is available online. The following immutable directory link identifies the exact Lean source used for the listings and the 118-declaration audit:

Browse the complete Lean proof at the frozen source revision

Source revision: `5dbda826fb9d385cf5528694f6908d28bc94054d`. The links in Table 7 point to that revision. They identify complete source files, not only the extracts printed here.

For an offline copy, download the **complete manuscript and Lean/code/data source bundle**. This stable download address tracks the current manuscript draft; the bundle's `source-manifest.json` and `artifact/MANIFEST_SHA256.json` identify its precise file contents. It includes the full Lean sources, the pinned toolchain and dependency manifest, source listings, axiom logs, experiments, results and validators. No dependency binaries are bundled. An additional **online proof guide** provides the same entry points.

After extracting the source bundle, enter `artifact/formalization/dyadic-rank` and run:

Complete source file	Contents and role
`CyclicLift/Basic.lean`	Voltage lifts, four-cycle predicates and native graph foundations.
`CyclicLift/Bipartite.lean`	Tanner incidence, bipartite coloring and short-cycle reduction.
`CyclicLift/LiftedProduct.lean`	Product support, explicit binary H_{X} and equality with its Tanner graph.
`CyclicLift/SixCharacterization.lean`	Necessity and sufficiency of the hexagon obstruction; supported by `ShortCycles.lean`.
`CyclicLift/GirthCharacterization.lean`	Constructive eight-cycle, full girth equivalence and rectangle-pair translation.
`CyclicStructure.lean`	All 651 lines of the general structural development: induced degrees, absorbing-support bijection and stopping bound.
`CyclicSourceQuestion.lean`	Public source-question corollaries.
`CyclicDecoding.lean`	General two-error bound and concrete induced-degree certificates.
`CyclicOptimized.lean`	Girth certificates for the selected finite candidates.

Table 7: Clickable roadmap to the complete formalization. Other library modules and the checking consumer are present in the linked directory.

```
lake -f lakefile-cyclic.toml exe cache get
lake -f lakefile-cyclic.toml build CyclicLift
lake -f lakefile-cyclic.toml env lean CyclicChecks.lean
lake -f lakefile-cyclic.toml env lean CyclicSourceQuestion.lean
lake -f lakefile-cyclic.toml env lean CyclicDecoding.lean
lake -f lakefile-cyclic.toml env lean CyclicOptimized.lean
lake -f lakefile-cyclic.toml env lean CyclicStructure.lean
```

The explicit `CyclicLift` build target avoids building an unrelated library named in the inherited alternate configuration. The five consumers emit the 75, 3, 8, 3 and 29 axiom reports respectively. From the `artifact` root, `python3 scripts/validate_cyclic_structure.py -recheck-lean` repeats the structural kernel check and checks exact audit coverage and source hashes. The package README gives the computational audit commands separately.

A.1 Complete later trajectory sources

The chronology's later Lean links use the immutable follow-up revision `c5561734e1b91c55911703ef4930f6e718624453`. The online proof guide lists every principal trajectory module, its role and reproduction commands. The refreshed bundle includes all imported local sources, including generated reachability data and concrete proof modules. Its `scripts/check_cyclic_paper_followups.py` builds local imports in dependency order before checking the merger, persistence, full trajectory and entry audit consumers. Historical build/axiom logs accompany the sources. These supplements do not alter the original 118-report audit or turn the statistical experiments into Lean probability-of-failure theorems.

References

[1] R. Freij-Hollanti, K. D. Morris and P. Šapokaitė, *Graphical Analysis of Lifted Product Code Constructions*, arXiv:2606.11987v1, June 10, 2026. https://arxiv.org/abs/2606.11987v1.

[2] M. P. C. Fossorier, *Quasi-Cyclic Low-Density Parity-Check Codes From Circulant Permutation Matrices*, IEEE Transactions on Information Theory **50**(8), 1788–1793, 2004. doi:10.1109/TIT.2004.831841.

[3] R. Smarandache and D. G. M. Mitchell, *Necessary and Sufficient Girth Conditions for Tanner Graphs of Quasi-Cyclic LDPC Codes*, arXiv:2105.03462v1, May 7, 2021. https://arxiv.org/abs/2105.03462v1.

[4] N. Raveendran and B. Vasić, *Trapping Sets of Quantum LDPC Codes*, Quantum **5**, 562, October 14, 2021. doi:10.22331/q-2021-10-14-562.

[5] B. Zhang, *On the Left-Right Lifted Product Quantum LDPC Code*, Master's thesis, Duke University, defended March 27, 2024, Section 3.3, Theorem 5. https://dukespace.lib.duke.edu/server/api/core/bitstreams/8f596ec8-b362-43da-bf61-f8c26f0c5edc/content.

[6] N. Raveendran, D. Declercq and B. Vasić, *On the Minimum Distances of Finite-Length Lifted Product Quantum LDPC Codes*, arXiv:2503.07567v1, March 10, 2025. https://arxiv.org/abs/2503.07567v1.

[7] K. Okada and K. Kasai, *High-Girth Regular Quantum LDPC Codes from Square-Base Hypergraph Products via CPM Lifts*, arXiv:2604.27817; v1 April 30, 2026; v3 September 11, 2026. https://arxiv.org/abs/2604.27817v3.

[8] J. Wang, L. Dolecek, Z. Zhang and R. Wesel, *Absorbing Set Spectrum Approach for Practical Code Design*, arXiv:1106.0057v1, May 31, 2011. https://arxiv.org/abs/1106.0057v1.

[9] J. Wang, L. Dolecek and R. Wesel, *The Cycle Consistency Matrix Approach to Absorbing Sets in Separable Circulant-Based LDPC Codes*, arXiv:1208.6094v2, December 5, 2012. https://arxiv.org/abs/1208.6094v2.

[10] E. McMillon, A. Beemer and C. A. Kelley, *Extremal absorbing sets in low-density parity-check codes*, Advances in Mathematics of Communications **17**(2), 465–483, 2023; published online March 31, 2021. doi:10.3934/amc.2021003.

[11] K. D. Morris, T. Pllaha and C. A. Kelley, *Absorbing sets in quantum LDPC codes*, Advances in Mathematics of Communications **23**, 234–254, 2026; published online March 3, 2026. doi:10.3934/amc.2026023.

[12] A. Abiad and N. Castriota, *Spectral and Additive Combinatorial Methods for Cycles and Absorbing Sets in Lifted-Product Quantum LDPC Codes*, arXiv:2607.13666v1, July 15, 2026. https://arxiv.org/abs/2607.13666v1.

[13] P. Panteleev and G. Kalachev, *Degenerate Quantum LDPC Codes With Good Finite Length Performance*, Quantum **5**, 585, 2021. doi:10.22331/q-2021-11-22-585.

[14] N. Raveendran, N. Rengaswamy, F. Rozpędek, A. Raina, L. Jiang and B. Vasić, *Finite Rate QLDPC-GKP Coding Scheme that Surpasses the CSS Hamming Bound*, Quantum **6**, 767, 2022. doi:10.22331/q-2022-07-20-767.

[15] N. Raveendran, J. Valls, A. K. Pradhan, N. Rengaswamy, F. Garcia-Herrero and B. Vasić, *Soft syndrome iterative decoding of quantum LDPC codes and hardware architectures*, EPJ Quantum Technology **10**, 45, October 19, 2023. doi:10.1140/epjqt/s40507-023-00201-1.

[16] C. Kelley, D. Sridhara, J. Xu and J. Rosenthal, *Pseudocodeword Weights and Stopping Sets*, ISIT 2004, Theorem III.1. `https://user.math.uzh.ch/kelley/isit2004_Chicago.pdf`.

[17] A. Rigby, J. C. Olivier and P. Jarvis, *Modified belief propagation decoders for quantum low-density parity-check codes*, Physical Review A **100**, 012330, 2019. `https://doi.org/10.1103/PhysRevA.100.012330`.

[18] S. Miao, A. Schnerring, H. Li and L. Schmalen, *Neural Belief Propagation Decoding of Quantum LDPC Codes Using Overcomplete Check Matrices*, arXiv:2212.10245, first posted December 20, 2022; v2 March 21, 2023. `https://arxiv.org/abs/2212.10245`.

[19] A. Tomasoni, S. Bellini and M. Ferrari, *Thresholds of absorbing sets in Low-Density-Parity-Check codes*, arXiv:1401.8168v1, January 31, 2014. `https://arxiv.org/abs/1401.8168`.